

A brief introduction to supervised, unsupervised, and reinforcement learning

Eduardo F. Morales^{*,†} and Hugo Jair Escalante^{*}

^{*}*Instituto Nacional de Astrofísica, Óptica y Electrónica, Puebla, Mexico*

[†]*Centro de Investigación en Matemáticas, Guanajuato, Mexico*

In recent years, artificial intelligence (AI) has been the subject of numerous articles in non-technical forums, news, blogs, videos, interviews, among others, that have grasp the attention of a general audience. The current hype promises autonomous vehicles, intelligent assistants, better health systems, and solutions for global challenges, such as hunger, climate change, treatment of diseases, to name a few. It also threatens with massive unemployment, larger gaps between rich and poor countries and people, up to human control by AI systems. Most of this hype has been as a consequence of recent advances in machine learning, and in particular, in deep neural networks.

Although the current hype, with high expectations and massive funding may very well end in a third AI winter, most people agree that in the long term more intelligent machines will be co-existing with humanity.

Considering recent events and the exponential growth in machine learning papers and researchers, writing a chapter in machine learning necessarily leaves out many relevant research work.

One of the goals of biosignal analysis is to be able to extract meaningful information from these signals, that could lead to better understanding of health and improved medical treatments. Machine learning techniques improve the performance of systems using data and, consequently, have been heavily used in biosignal processing. In this chapter, we will superficially cover some of the most common machine learning techniques, providing the main ideas behind them and pointing out some relevant references. For an in depth treatment, there are several machine learning books the interested reader can refer to (e.g., [21], [40], [2], [3], and [9]) and some books for specific ML areas, such as Reinforcement learning [36], Deep neural networks [11], or Decision trees [28], among others.

The chapter is organized as follows: First a brief history of the area is presented. We then describe some of the algorithms used for supervised, unsupervised, and reinforcement learning. In the last section, we provide some conclusions.

6.1 Brief history of the area

Since the development of computers in the late 40s and the creation of the area of artificial intelligence in 1956, researchers have always considered the possibility of creating intelligent machines. Intelli-

gence is normally associated with the capacity of learning, so it is not surprising that machine learning is one of the core areas of AI. In 1950, Turing proposed what has been called the Turing test [38], where a machine could be considered intelligent if it could fool most of the time a human interrogator in differentiating a machine from a human. For some people (e.g., [23]), and specially from a machine learning perspective, the most interesting part of the paper, is in its second half, where Turing suggested that instead of creating a machine to pass the test, it would make more sense to create a *child machine*, which could be taught to become an “adult” machine capable of passing the test. Turing then suggested several possible machine learning techniques based on imitation learning, learning through rewards, and even evolutionary techniques, among others.

Since the beginning of computers, researchers started to create basic machine learning algorithms from different perspectives, many of which followed some of the ideas suggested by Turing. Some tried to emulate in an abstracted and simplified way the human brain, through artificial neural networks. Possibly the first paper on machine learning is that of McCulloch and Pitts [18] that shows how to emulate logic propositional gates using neural networks. Some years later, convergence tests arrived for simple neural networks or perceptrons (e.g., Rosenblatt [29]). This research line has continued to evolve up to our days, where the most recent models use several layers, called deep learning, and is responsible for the most recent hype in AI.

Other researchers followed a simple learning scheme based on punishments and rewards. This area, called reinforcement learning [36], had also some early success with AI. Samuel’s checkers program [32], the pole-balancing problem [19], and, more recently, for playing backgammon [37]. In RL, an agent learns the best action to take at any time to maximize its expected accumulated reward. Recently, the merging of RL and deep learning has produced systems that can beat humans in Atari games [22], and were able to recently beat the champion in Go [34].

Other researchers explored learning symbolic representations, such as Hunt [15] and Feigenbaum [8]. One of the mostly used algorithm in machine learning, which was derived from this line of research, is learning decision trees [28]. The most recent and powerful algorithms in this line of research create ensembles of trees, such as random forest [5] and XGBoost [7].

6.2 Machine learning

Machine learning studies and develops algorithms able to improve its performance with experience. Its emphasis is in inducing models without or with little human intervention, using mainly data. When people try to solve a problem, they normally create a model or a program able to solve it. However, in many cases, it is difficult to properly formalize the problem; there are not necessarily experts available, although there might be plenty of data. For instance, if we want to create a program to recognize people by analyzing their faces, we need to process the image, create a program to identify distinctive features, which we first need to know or consult an expert, and then create a program able to compare these distinctive features among different faces. With a machine learning algorithm, we simply get the images and let the system to automatically generate a model from the data. This clearly opens a large number of possible opportunities for useful applications, where it is difficult to properly formalize or program a solution for some problems that have available data.

The recent success and hype of these algorithms obey four key factors: (i) availability of large quantities of data, (ii) more powerful machines with larger storage capacity and faster processing units,

(iii) availability of open source code, allowing a general usage of these techniques, and (iv) massive funding due to the increasing interest in the area from governments and industries.

The machine learning area can be classified in different ways. Some authors [9] classify the area in terms of the models that they produce: (i) geometric, (ii) probabilistic, and (iii) logic models. ML algorithms can also be classified by the task that they are trying to solve: (i) create descriptions, summaries, identify prototypes and anomalies in data, (ii) segment/group data in clusters, (iii) create prediction models to output, given some input, a discrete value for classification problems or a continuous value for regression models, (iv) detect dependencies between variables, which could be probabilistic, associative, functional, causal, and (v) learn control policies or plans to solve sequential decision problems. Considering the contents of this chapter, we will follow a more traditional, although not necessarily better, approach for a wider view of ML, and classify the area in three categories: (i) supervised, (ii) unsupervised, and (iii) reinforcement learning.

6.2.1 Supervised learning

Most of the machine learning algorithms build prediction models that, given some inputs, produce outputs in the form of discrete values, for classification tasks or continuous values for regression tasks. These algorithms are part of what is known as supervised learning.

Formally, in supervised learning, we have available a dataset $\mathcal{D}$, formed by N pairs of d -dimensional samples, $\mathbf{x}_i \in \mathbb{R}^d$, and labels $y_i \in Y$,¹ that is: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i \in 1, \dots, N}$. The goal of supervised learning is to find a function $f: \mathbb{R}^d \rightarrow Y$ mapping inputs to outputs, i.e., $y_j = f(\mathbf{x}_j)$, such that it can generalize beyond $\mathcal{D}$. Usually, $\mathcal{D}$ is split into training and validation partitions, hence the goal is learning f from $\mathcal{D}$ such that label predictions can be made for any other instance sampled from the same underlying distribution as $\mathcal{D}$. If we denote $\mathcal{T}$ the *test* set, formed by instances coming from the same distribution as $\mathcal{D}$ but that do not appear in such set, $\mathcal{T}$ can be used to evaluate the generalization capabilities of f .

6.2.1.1 Classification

The goal of a classifier is to automatically induce a model from data, such that given a description of an instance it can produce as output a class label. For instance, given information of the symptoms of a patient, a classifier can predict a disease, or given information from variables of a production line, detect a failure.

There is a large number of possible classifiers, such as decision trees, support vector machines, instance-based learning classifiers, among others. In this section, we are going to briefly describe some of them.

6.2.1.2 Decision trees

In a decision tree, each node represents a test over an attribute, and its branches are the possible outcome of the test. The leaves of the tree have the classes that we want to predict. The algorithm uses a heuristic to select the best current attribute, which partitions the data over the different branches. Once an attribute is selected, the algorithm is applied to the data of each branch until (i) a class label

¹ If Y is a finite set of categories, we have a classification task, and if $Y \in \mathbb{R}$ we have a regression task.

is assigned to a branch, where the examples of a branch belong to a single class, (ii) there are no more examples, and a class label is assigned to the majority class of the node, and (iii) there are no more attributes, and the class label is assigned to the majority class of the examples of that branch.

In the classification stage, the decision tree is traversed, considering the values of the attributes of the example to be classified. When reaching a leaf, the associated class label is returned.

There are different heuristics that have been proposed to select the best attribute; the two most well known are information gain and the Gini index. The considered heuristic should take a maximum value when the attribute perfectly distinguishes between positive and negative examples (in case of binary problems), and a minimum value when it is not relevant. One possibility is to use a measure based on Shannon and Weaver entropy value [33]. The information content measures how (im)pure is a collection of examples in terms of their class value. The amount of information (measured in bits) received with respect to the occurrence of the event is inversely proportional to the probability of such event. If there are v_i possible outcomes with probabilities $P(v_i)$, the information content is

$$I(P(v_1), \dots, P(v_n)) = - \sum_{i=1}^n P(v_i) \log_2 P(v_i).$$

In the case of trees, we want to estimate the probabilities of classes, which can be obtained with the proportion of positive and negative examples. If there are p positive and n negative examples, then

$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}.$$

A single attribute normally cannot give us that amount of information, however, we can estimate how much information we need after using an attribute. Each attribute A , divides the examples in subsets $E_1, E_2, \dots, E_v$, depending on the v values of the attribute. Each subset E_i has p_i positive and n_i negative examples, so for each branch we need $I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$ amount of information to produce an answer. A random example has the i -th value of the attribute A with probability $\frac{p_i+n_i}{p+n}$. So in average, after testing attribute A , we need

$$E(A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right).$$

The information gain that we obtain when selecting an attribute is therefore

$$InfoGain(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - E(A).$$

The information gain of A says how many bits we saved when we try to return the class of an example, given that we know the value of attribute A . This information gain measures how well the attribute divides the examples according to their classes, and the attribute with larger information gain is selected.

Another popular heuristic is the Gini index used in CART [6] and shown below:

$$Gini(t) = 1 - \sum_{j=1}^m (p(j|t))^2,$$

where $p(j | t)$ is estimated as the relative frequency of the class j in t . The idea is to minimize this index when selecting an attribute. For that, the index is evaluated on each branch of the attribute, taking into account the proportion of examples of each branch. If there are k branches:

$$Gini_A = \sum_{i=1}^k \frac{n_i}{n} Gini(k),$$

where n_i are the examples of the branch, and n are the examples of the node.

The original decision tree algorithm assumed only nominal attributes, no noise, and no missing values. Several extensions were proposed in the literature to deal with them. For continuous attributes, the examples are ordered in ascending (or descending) order, adjacent examples with different class values are identified, and their midpoint is selected. Information gain is evaluated over every division point, as well as the information gain of nominal attributes, and the one with larger information gain is selected. In this case, numerical attributes can appear more than once in a single branch of a tree.

In case of noise, it is common to create models that overfit the data. Given a hypotheses space H , a hypothesis $h \in H$ is said to *overfit* the training data if there is another hypothesis $h' \in H$, such that h has less errors than h' in the training data, but h' has less errors than h in the whole distribution of the examples.

Different strategies have been proposed to deal with noise. In the case of decision trees, the options have been *pruning* (or *pre-pruning*), changing the stopping criterion to prune branches while building a tree, and *post-pruning*, pruning branches after the decision tree has been constructed. The mostly used technique is post-pruning, using an estimate of re-substitution and statistical tests.

Decision trees have been extended to deal with regression problems in the form of regression trees (store an average value of each leave) and model trees (use linear regression to predict the values of the class). In this case, instead of information gain, the algorithms use *standard deviation reduction* to select the most relevant attributes:

$$SDR = std(T) - \sum_i \frac{T_i}{T} \times std(T_i),$$

where $T_1, T_2, \dots$ are the subsets of data produced after selecting an attribute, and *std* is the standard deviation.

6.2.1.3 Support vector machines

Another well known algorithm is support vector machines (SVM) [4]. This algorithm displaced the existing neural network systems at that time, as it produced very competitive performance, with much smaller training times, and having clear mathematical foundations based on geometry and theoretical bounds on performance. SVMs also make popular the *kernel trick* and the development of kernel machines.

One way to build a classifier is to create a plane or hyperplane to divide the positive and negative examples in the input space. This can be posed as a geometric problem, using vectors and dot products. Finding an adequate hyperplane is posed as an optimization problem, where the idea is to find the plane that best divides the classes. We can define a similarity measure k such that given two instances, x and

x' , k returns a real value that characterizes their similarity:

$$k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R},$$

$$(x, x') \rightarrow k(x, x')$$

with:

$$k(x, x') = k(x', x),$$

where we refer to function k as a *kernel*.

Given a training set: $\{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\} \in \mathbb{R}^M$ and $y_1, y_2, \dots, y_N \in \{-1, +1\}$, we want to find the distance between hyperplanes

$$\vec{w} \cdot \vec{x} + b = -1, \vec{w} \cdot \vec{x} + b = +1$$

or equivalently

$$\vec{w} \cdot \vec{x} + (b + 1) = 0, \vec{w} \cdot \vec{x} + (b - 1) = 0.$$

We know that $D = \frac{b_1 - b_2}{\|\vec{w}\|}$, so $D = 2/\|\vec{w}\|$. Since we want to maximize the separation between planes, we can minimize $\|\vec{w}\|$, or equivalently, for convenience, minimize $\frac{1}{2}\|\vec{w}\|^2$.

We also need to place restrictions so that all of the examples in the training set are well classified:

$$\vec{w} \cdot \vec{x}_i + b \leq -1, \text{ if } : y_i = -1$$

$$\vec{w} \cdot \vec{x}_i + b \geq +1, \text{ if } : y_i = +1.$$

Or, equivalently,

$$y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1$$

In summary, the optimization problem that is solved by SVMs is

$$\min \tau(\vec{w}) = \frac{1}{2}\|\vec{w}\|^2$$

s.t.

$$y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1, \text{ for } : i = 1, \dots, N.$$

Given a new instance, $\vec{x}$, the class is given by

$$f(\vec{x}) = \text{sign}(\vec{w} \cdot \vec{x} + b).$$

The linear formulation can be solved using Lagrange multipliers, and the optimization problem then becomes

$\min \tau(\vec{w}) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$ subject to: $\alpha_i \geq 0$ and $\sum_{i=1}^N \alpha_i y_i = 0$ (restrictions).

The solution is given by $f(\vec{x}) = \text{sign}(\sum_{i=1}^N \alpha_i y_i \vec{x}_i \cdot \vec{x} + b)$.

With this formulation, we do not need to access to the original data, but only to the involved dot products. Also the number of free variables is bounded by the number of support vectors (those samples for which $\alpha_i \neq 0$) and not by the number of attributes.

When the data is not linearly separable, we can use slack variables, leaving soft margins. The slack variable can be seen as the distance that separates the instance from the hyperplane which is wrongly classified (or 0, well classified). Similarly, data which cannot be linearly separated in the attributes' space, but may be linearly separable in a higher dimensional space, obtain with a kernel.

Extensions to the original SVM formulation have been developed for model selection, multiclass problems, for regression problems, for detecting outliers, for clustering, and for selecting attributes, among others.

6.2.1.4 Instance-based learning

Instance-based algorithms, rather than trying to induce a model, store the examples and use them to classify new instances, by searching for the most similar examples. The most well known algorithm in this research area is k-nearest neighbors (k-NN). Given a new instance to classify, the algorithm searches its dataset for the k elements which are more similar, under a similarity measure, to the instance, and assigns the majority class of the selected nearest examples or neighbors.

For regression problems, the mean of the outputs of the nearest neighbors can be used:

$$f(x_q) = \frac{\sum_{i=1}^k f(x_i)}{k}.$$

An extension is to use a *weighted average*, taking the average of the outputs weighted inversely proportional to their distance. For nominal classes:

$$f(x_q) = \operatorname{argmax}_{v \in V} \sum_{i=1}^k w_i \delta(v, f(x_i)),$$

where V is the set of classes, w_i can be, for instance, $w_i = \frac{1}{d(x_q, x_i)^2}$ (if the distance is 0, then $w = 0$) and $\delta(a, b) = 1$ when $a = b$.

For continuous classes:

$$f(x_q) = \frac{\sum_{i=1}^k w_i f(x_i)}{\sum_{i=1}^k w_i}.$$

One possible problem with similarity-based learning is when there is a large number of irrelevant attributes that may dominate over the classification (e.g., two relevant attributes with 20 irrelevant attributes will not have a large influence in the decisions). One option is to give more weight to the more relevant attributes. These weights could be selected by the user or tuned with known examples trying to minimize the classification error. Also a feature selection algorithm could be used to reduced irrelevant attributes.

There is no training time in k -NN, but it has a memory cost for storing all the instances, and it is computationally costly to produce a class as it compares the instance to be classified with all the stored examples. Clearly this is problematic with large databases, so it makes sense to use techniques

to reduce the number of training examples without compromising the effectiveness of the classifier. Some approaches include the following:

- **Selection of prototypes:** Select a subset of training instances
- **Generation of prototypes:** Generate synthetic points that cover the input space

For the first case, a simple strategy is to generate a centroid or k prototypes for each class $v \in V$. For class v , the prototypes can be obtained running k -means (or some other clustering technique). For the latter case, techniques combining instances to learn prototypical instances (see e.g., [10]).

An additional practical element, has to do with storing all the examples. Sometimes a pre-processing step is performed to build kd -trees, where the instances are distributed in a tree considering their similarity.

A closely related research area is case-based reasoning (CBR), where each example, or case, is a representation of a problem and its solution. When a new case is presented to the system, it searches in its case database for the most similar cases and uses the solution or an adaptation of that solution to solve the new case. CBR uses a much richer representation for each instance (called case). CBR solves new cases by adapting previous solutions used in similar problems. Each case, is normally associated with the description of the problem, a description of its solution strategy, and information of the obtained results. In this framework, the similarity measures are normally more complex, and the combination of solutions is not trivial, may involve background knowledge, search strategies, and additional reasoning mechanisms.

6.2.2 Dependency analysis

Some algorithms automatically find dependencies between variables, which can be probabilistic, functional, associative, and causal. With these dependency models, one can estimate the value of a variable given the values of other variables. For instance, what is the probability of rain, given information of atmospheric pressure and temperature, or how frequent and with what confidence people that buy milk and bread, also buy butter and jam.

One of the most common techniques to capture probabilistic dependencies are probabilistic graphical models, and in particular, Bayesian networks. Bayesian networks capture a joint probability distribution of stochastic variables, represented by a directed acyclic graph, where each node represents a stochastic variable and the arcs probabilistic dependencies among variables. Bayesian networks perform probabilistic reasoning using Bayes rule, and can evaluate the most probable value of some variables, given the values of other variables.

In general, Bayesian learning is at the core of machine learning. Under a Bayesian perspective, we can frame the machine learning problem as trying to find the most probable hypothesis h , given the data D . We can express this in terms of Bayesian theorem. If $P(D)$ = probability *a priori* of the data, $P(D | h)$ = probability of the data given a hypothesis h , what we would like to estimate is $P(h | D)$, the *posterior* probability h given the data. This can be expressed with the Bayes theorem as

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}.$$

We can search for the most probable hypothesis or MAP (*maximum a posteriori hypothesis*):

$$\begin{aligned} h_{MAP} &= \operatorname{argmax}_{h \in H} (P(h \mid D)) \\ &= \operatorname{argmax}_{h \in H} \left(\frac{P(D \mid h)P(h)}{P(D)} \right) \\ &\approx \operatorname{argmax}_{h \in H} (P(D \mid h)P(h)), \end{aligned}$$

since $P(D)$ is independent of h .

If we suppose that the hypotheses are equally likely, we end up with the *maximum likelihood* hypothesis:

$$h_{ML} = \operatorname{argmax}_{h \in H} (P(D \mid h))$$

The simplest classifier using Bayesian networks is naïve Bayes. It is used to find the most probable class value ($v_j \in V$), given values of the attributes of an example:

$$v_{MAP} = \operatorname{argmax}_{v_j \in V} (P(v_j \mid a_1, \dots, a_n))$$

With Bayes:

$$\begin{aligned} v_{MAP} &= \operatorname{argmax}_{v_j \in V} \left(\frac{P(a_1, \dots, a_n \mid v_j)P(v_j)}{P(a_1, \dots, a_n)} \right) \\ &= \operatorname{argmax}_{v_j \in V} (P(a_1, \dots, a_n \mid v_j)P(v_j)) \end{aligned}$$

$P(v_j)$ can be estimated from the frequency of the classes, however, for $P(a_1, \dots, a_n \mid v_j)$, we have very few data. Naïve Bayes assumes that the attribute values are conditionally independent between each other, given the class values, so $P(a_1, \dots, a_n \mid v_j) = \prod_i P(a_i \mid v_j)$. Consequently

$$v_{NB} = \operatorname{argmax}_{v_j \in V} \left(P(v_j) \prod_i P(a_i \mid v_j) \right).$$

The values for $P(a_i \mid v_j)$ can now be easily estimated with the frequency of the training data.

Aiming for more general models, we would like to induce full Bayesian networks. This process is naturally divided into two parts:

1. **Parametric learning:** Given a Bayesian network structure, obtain the *a priori* and conditional probabilities required for the network. This is normally done counting frequencies of values, although more sophisticated methods are used when there are missing values or hidden variables.
2. **Structural learning:** Obtain the structure of the Bayesian network, i.e., the dependency and independency relations between the observed variables. There are two main methods for learning Bayesian networks: (i) based on search and heuristic measures and (ii) based on independence tests.

In structural learning, methods that use search have two main components: an evaluation measure that provides a value of how good is the current structure with respect to the data (e.g., BIC, MDL,

etc.) and a search strategy that considers different structures and continues searching until a termination criterion is met.

Methods based on independence tests tried dependency tests between subset of variables. One of the most well known algorithm is PC [35], which first obtains an undirected graph, and then it directs the links between nodes. PC starts with an undirected and completely connected graph and determines the conditional independence of each pair of connected variables, given incremental subsets of the other variables, using a conditional entropy measure.

Another way to obtain relations between variables is through association rules. The classic algorithm is called Apriori [1], which first searches for all the attribute-value pairs, called *itemsets*, that have a minimum support from the search. This search is performed in an incremental way, first searching for pairs of itemsets, then considering triplets, and so on, until there is no more subset of attribute-value pairs with the minimal support. If an itemset has not the minimal support, all its supersets are eliminated. For each itemset, all the possible rules that satisfy a minimum confidence value (support of the item-set divided by the support of the antecedent of the rule) are built. These rules are constructed considering all the possible subset combinations of antecedents and consequents obtained from the itemset. This area is closely related with other machine learning areas that discover frequent patterns, subgroup discovery, etc.

Both Bayesian networks and association rules can be easily modified to produce classifiers, where the aim is to predict the value of a single variable (the class). Other approaches have been suggested to find functional dependencies and more recently causal models [27]. A causal model represents cause-effect relations between variables. This allows us to understand how some variables obtain their values and can be used to predict values after some other variables are intervened. This is more expressive than Bayesian models and can be used for counterfactual reasoning, e.g., what would have happened if a variable has taken some value.

6.2.2.1 Regression and deep neural networks

The goal in regression algorithms is to automatically induce, from data, a model that produces a continuous output, given an example as input. For instance, predicting the temperature of an industrial process, given the values of the variables involved. Some of the most used techniques are neural networks, lineal regression, locally weighted regression, Gaussian processes, among others.

Possibly the most well known technique used for regression is neural networks. It is the recent advances in neural networks that have produced the current hype in AI. A classical neural network is a directed graph organized in layers, in which all the nodes in one layer are connected to all the nodes in the next layer. Each link is associated with a weight, and each node in a layer receives a weighted sum of all the outputs of the nodes of the previous layer. This weighted sum is passed through an activation function (a continuous, nonlinear, differential function), which produces the output of that node. This output and all the outputs of all the nodes of the same layer are propagated to the new layer with their respective weights. This process continues until producing the final outputs. The weights in the network are randomly initialized, and the learning process consists of adjusting such weights to match the outputs from the network with the desired outputs of the examples. The adjustment is gradually performed changing weights in the direction that produces the maximum descent in the error surface. This is done by taking the negative derivative of a loss (or error) function with respect to the weights and using a small learning rate. The error between the predicted and real value is propagated

back through the network until there is little change in the weights, or the performance of the network is considered acceptable.

Neural networks were popular during the initial years of AI; they lost their popularity at the end of the 60s with a book that show their limitations at that time [20]. The area gained acceptance again with the arrival of the back-propagation algorithm in the mid 80s [30], and lost again some impetus with the arrival of SVMs. They recently gained a lot more popularity with the introduction of deep neural networks [13], which has been possible due to the availability of large amounts of data, processing speed with the introduction of GPU, and some technical improvement over the back-propagation algorithm.

In its simplest form, neural networks have two components:

- A lineal component (input function) which is a weighted sum of all the inputs to a node

$$in_i = \sum_j w_{j,i} a_j = \mathbf{w}_i \cdot \mathbf{a}_i,$$

where $\mathbf{w}_i$ are the weights arriving to unit i and $\mathbf{a}_i$ are the activation values of the inputs to unit i .

- A non-linear differentiable function (activation function). The new activation value (after applying function g):

$$a_i \leftarrow g(in_i) = g\left(\sum_j w_{j,i} a_j\right).$$

Several activation functions have been proposed, step-size, sign, sigmoid, and more recently, ReLU ($ReLU(x) = \max(0, x)$) or some variants, such as *absolute value rectification*, *leaky ReLU*, *parametric ReLU*, or *PReLU*, among others.

To adjust the neural network weights (θ) the algorithms used gradient descent. For continuous outputs, the error can be represented with the mean square error:

$$E(\theta) = \frac{1}{2} \sum_i (t_i - o_i)^2,$$

where t_i is the real value, and o_i is the predicted value. To minimize the error, we take the gradient to find the direction of the greater increment and use its negative value:

$$\theta \leftarrow \theta + \Delta\theta,$$

$$\Delta\theta = -\alpha \nabla E,$$

$$\begin{aligned} \frac{\partial E}{\partial \theta_i} &= \frac{\partial}{\partial \theta_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial \theta_i} (t_d - \vec{\theta} \cdot \vec{x}_d) \\ &= \sum_{d \in D} (t_d - o_d) (-x_{i,d}). \end{aligned}$$

So

$$\Delta\theta_i = \alpha \sum_{d \in D} (t_d - o_d) x_{i,d}.$$

This can be done incrementally, for each example, using all the examples, or using randomly selected examples or mini-batches or *stochastic gradient descent* (SGD), which is currently the most common technique.

In case of nominal classes, the error is normally expressed using cross-entropy, which is equivalent to reduce the Kullback–Leibler divergence.

In many cases, the loss function includes a regularization term to provide more stability to the learning process. Despite the wide availability of data, sometimes it is not enough to properly train a deep neural network, and it is also common to perform *data augmentation* (transformations on the original data to generate new data). For faster training, it is also common to do *dropout*, which occludes some randomly selected units during training.

Recently there has been a quick development in optimization algorithms. A common problem when using gradient descent are the local minimum, *plateaus*, cliffs, far dependencies, etc. In addition to stochastic gradient descent (SGD), several improvements have been proposed, such as stochastic gradient descent with momentum and with Nesterov momentum, AdaGrad, RMSProp, RMSProp with Nesterov momentum, Adam (adaptive moments), and second-order methods, among others.

A large number of network architectures have emerged in recent years. There are restricted Boltzmann machines, deep autoencoders, convolutional neural networks, recurrent neural networks, etc.

Convolutional neural networks take a patch of the image of the size of the kernel or the filter and perform a piece-wise multiplication. The kernel is slid over all the image, producing what is called a feature map. Several kernels are used; their outputs are processed by an activation function (e.g., ReLU), and a pooling process replaces the output by a statistic, e.g., max-pooling returns the maximum value on the resulting patch. This is applied again in subsequent layers, and at the end, the information is transformed into a single vector, where normal neural network layers are applied to obtain a desired output. Convolutional neural network became popular after achieving significant improvement over the current systems in the ImageNet LSVRC-2010 contest with 1000 classes [17].

Another popular type of neural network is recurrent network, where connections form a temporal sequence. Contrary to traditional feedforward networks, they receive as input, in addition to the current input, what they perceived previously. Perhaps the most successful recurrent network is long short-term memory (LSTM), which prevents backpropagated errors from vanishing (or exploding) via a gated cell, which learns what information to block or pass [14]. Other developments include GRUs and transformers.

Generative adversarial networks (GANs) were created to estimate generative models via an adversarial process. There are two models: a generative model that captures the data distribution, and a discriminative model that estimates whether the data came from the training data or from the generative model. The generative model tries to maximize the probability that the discriminative model makes mistakes. This process converges when the generative model fits the training data distribution and the discriminative model has 0.5 of probability all the time [12]. They have been involved in the generation of fake news.

There are much more deep learning models and algorithms. In general, they normally require large amounts of data and processing time. Researchers are trying to reduce these requirements and also seeking to produce explainable models, which can be easily reproduced.

6.2.3 Unsupervised learning

Methods that automatically segment data are known as clustering; it is also common to refer to them as unsupervised learning. The core idea behind these algorithms is to automatically create groups from the data such that elements in the same group are similar, under certain criteria, to each other, and dissimilar, under the same criteria, to elements of other groups. For instance, data of clients could be clustered in groups for segmented publicity campaigns. The clusters could be disjoint or have overlaps; they could be organized in hierarchies; could be exhaustive; the membership to a class could be probabilistic, etc. The similarity measure is normally defined in terms of the proximity between elements in a multidimensional space, and there is a larger number of similarity measures used to group elements. If the data is numeric, normally measures, such as Euclidean, Manhattan, or Minkoski are used. There are similarity measures for binary, nominal, ordinal, non-linear scalars, and mixed variables. If the data is not described by numbers, normally the measures are not longer metrics.

Occasionally, some type of attributes or variables can be considered more important than others, and this can be incorporated by giving more weight to these attributes in the used similarity measure.

Several strategies to form clusters have been proposed in the literature. Methods based on partitions build k clusters. They create an initial partition, which is incrementally refined until a termination criterion. The most popular algorithm in this category is k -means, although there are others, such as CLARA and CLARANS. Some methods create hierarchies, either bottom-up (starting with a group for each element and joining similar groups until a single group is formed or another stopping criterion is met, e.g., Agnes, Birch, Cure, Rock) or bottom-up (starting with a single group and dividing it into smaller groups until a single element represents a group or another stopping criterion is met, e.g., Diana, Mona). Other methods are based on density, forming groups according to their neighbors within certain threshold value, e.g., Dbscan, Denclue. Some methods are based on dividing the space in grids at different levels (e.g., Sting, Clique), based on statistical models (e.g., COBWEB, AutoClass), based on graph theory (e.g., Chameleon), Delaunay triangulation graph (DTG), highly connected subgraphs (HCS), clustering identification via connectivity kernels (CLICK), cluster affinity search technique (CAST). Clusters can be formed using combinatorial search (e.g., genetically guided algorithm (GGA), TS clustering, SA clustering), based on fuzzy techniques (e.g., fuzzy c-means (FCM), mountain method (MM), possibilistic c-means clustering algorithm (PCM), and fuzzy c-shells (FCS)). Neural networks have been also used for clustering (e.g., learning vector quantization (LVQ), self-organizing feature map (SOFM), ART, simplified ART (SART), hyperellipsoidal clustering network (HEC), self-slitting competitive learning network (SPLL)).

As previously mentioned, one of the most well known clustering algorithm is k -means. Given a number of clusters (k) to form, it randomly selects k elements and evaluates their similarity with the rest of the data. Each instance is assigned to the closest k element. With the data associated with each k , a centroid or “average” element is constructed. All the elements of the data evaluate their similarity with the new centroids, from which possibly new clusters are formed. This process is repeated until the centroids do not change.

Another popular algorithm is DBSCAN, which finds high-density regions. The density of a point p is the number of points (data) within a circle of radius ϵ . A region is considered dense if it has at least a minimum number of points. The neighbors of a point p are

$$N_{\epsilon}(p) = \{q \in D | dist(p, q) \leq \epsilon\}$$

A point is called a core point if $|N_{\epsilon}(p)| \geq MinPts$.

A point a is *directly density reachable* from a point b if

1. $|N_\epsilon(b)| \geq \text{MinPts}$ (i.e., is a core point),
2. $a \in N_\epsilon(b)$ (a is in the ϵ vicinity of b).

A point a is *density reachable* from point b if in a chain of point $b_1, b_2, \dots, b_n$, $b_1 = b$, $b_n = a$, and b_{i+1} is *directly density reachable* from b_i . A point a is *density connected* from point b if there is a point c such that a and b are *density reachable* from c . BDSCAN starts with an arbitrarily point and forms clusters using density connected points.

Several measures have been proposed to evaluate the quality of the clusters. They are divided into internal measures, which consider the information on the constructed clusters (e.g., Davies–Bouldin, Dunn index, Silhouette coefficient) and external measures, which compare the created clusters against known groups (e.g., purity, Rand index, F-measure, Jaccard index, Dice index, Fowlkes–Mallowd index).

Some challenges in the area are how to deal with different representations in the data, how to create clusters of arbitrarily shapes, how to include restrictions, and how to make the clusters interpretable, among others.

6.2.4 Reinforcement learning

When we think about learning, one of the ideas that comes to our minds is learning through interaction with the environment. This interaction provides us with information about the consequences of our actions that could be used to achieve goals. The machine learning technique that uses interactions to learn how to behave in an environment is reinforcement learning (RL). In RL, the objective is to learn how to map states to actions (a policy) to maximize the total expected reward. The rewards can be given at final and at intermediate states. The agent seeks useful experience about states, actions, state transitions, and rewards to learn how to behave optimally. Contrary to other machine learning techniques the evaluation of the system occurs concurrently with the learning process. RL follows a trial and error process; there are delayed rewards, and there is a balanced between exploration and exploitation.

In RL, the agent has to decide which action to take on each state. This sequential decision process can be modeled by a Markov decision process or MDP. An MDP is a tuple $M = \langle S, A, \Phi, R \rangle$ characterized by the following:

- A finite set of states S , ($s_i \in S, i = \{1, \dots, n\}$)
- A finite set of actions A , which may depend on each state ($a_j(s_i), j = \{1, \dots, m\}$)
- A reward function (R), which defines the goal and maps each state-action pair to a number showing how good is that state ($f(s, a) \rightarrow \mathcal{R}$)
- A state transition function $\Phi(s'|s, a)$ ($\Phi : A \times S \rightarrow S$), which gives the probability of reaching state $s' \in S$ when taking action $a \in A$ in state $s \in S$

Two additional elements are as follows:

- The policy (π) that defines how the system behaves at certain time. It is a mapping (sometimes stochastic) between states and actions ($\pi(S) \rightarrow A$)

- Value function (V^π / Q^π), which is the total expected reward that the agent can receive from state s ($V^\pi(s)$), or from state s taking a particular action a ($Q^\pi(s, a)$) while following a policy π

There are three main approaches for solving MPDs:

1. Dynamic programming (DP), which assumes that the transition and reward functions are known and are used to evaluate the value function. In principle, these methods used bootstrapping (estimating values using estimations of other values), for instance, $V(s) \leftarrow \sum_{s'} \mathcal{P}_{ss'}^{\pi(s)} [\mathcal{R}_{ss'}^{\pi(s)} + \gamma V(s')]$, and then trying actions which are different from the current policy, but which are higher values to improve the policy, as in policy iteration, or directly using the best action for the value function

$$V_{k+1}(s) = \max_a \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V_k(s')]$$

as in value iteration;

2. Monte Carlo (MC), which uses simulations to gather statistics that approximate the value functions;
3. Temporal differences (TD), which use bootstrapping, such as DP, but the agent does not need to know the transition and reward functions, like (MC).

The most well known algorithm in reinforcement learning is Q-learning [39]. The algorithm is shown below:

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

 Initialize s

 Repeat (for each step in the episode):

 Select an action a in s using a policy derived from Q

 (e.g., ϵ -greedy which selects the action with the best

 Q-value but with a small probability (ϵ) randomly

 selects an action)

 Take action a , observe r, s'

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$;

 until s is terminal

There are other algorithms, such as SARSA, expected SARSA, and double Q-Learning, among others. Also researchers have designed algorithms that are between MC and TD (such as n-step TD and eligibility traces), or between DP and TD (which learn models while exploring the environment, such as Dyna-Q and prioritized sweeping).

In large state-action spaces storing the value functions in tabular forms is impractical. Several approaches have been suggested based on abstractions, hierarchies, relational representations, and function approximations. Also additional aids can be used to help the agent to converge faster to the optimal policy, such as reward shaping, using programming by demonstration, and providing user's feedback.

Recently this area has received a lot of attention when used in combination with deep neural networks. This allows agents to learn directly from images, which can be used in autonomous vehicles

(drones, mobile robots, and cars) and in games. We can approximate value functions, the policy function, or both. In this case, we can use all the recent developments in deep neural networks based on gradient descent, where the parameters of the network for instance, for the value function, could be updated as

$$\vec{w} \leftarrow \vec{w} + \alpha [R + \gamma \hat{v}(S', \vec{w}) - \hat{v}(S, \vec{w})] \nabla \hat{v}(S, \vec{w}).$$

In this case, we have a semi-gradient, since we do not have the real value function to evaluate the error.

If we want to directly approximate a function for the policy, the gradient policy theorem can be used, and the parameters of the policy function are updated as

$$\Theta_{t+1} = \Theta_t + \alpha (G_t - b(S_t)) \frac{\nabla_{\Theta} \pi(A_t | S_t, \Theta)}{\pi(A_t | S_t, \Theta)},$$

where

- G_t is the total return obtained from that state
- A_t is the action selected by the policy
- $b(S)$ is a base function to reduce variance; a natural candidate is the value function $\hat{v}(S_t, w)$, where w are their parameters

Some of the challenges when trying to use deep learning (DL) with RL is that DL requires a large number of examples, RL has sparse and delayed rewards, DL assumes that the data is independent, and its distribution does not change during learning.

DQN was the first algorithm to successfully combine deep neural networks with reinforcement learning [22]. It was used to learn how to play Atari games, reaching higher performance than humans in 29 out of the 46 games. DQN used two main ideas; one of them was to store experiences of the agent while exploring the environment ($e_t = (s_t, a_t, r_t, s_{t+1})$) in a database $\mathcal{D} = e_1, \dots, e_N$ and updating the parameters of the neural network for the Q function sampling $\mathcal{D}$ (*experience replay*), the equivalent to mini-batch. The other idea to reduce variance was to fix a copy of the network for some episodes and interchange the networks after that:

$$\nabla_{\theta_i} L(\theta_i) = \mathbb{E}_{s,a,r,s'} \left[(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i)) \nabla_{\theta_i} Q(s, a; \theta_i) \right],$$

where θ_i^- estimates Q with fixed weights, and θ_i is the network that is updating its weights. After a certain number of steps, the networks are interchanged.

After DQN, several extensions have been suggested, such as prioritized experience replay, double DQN, deep deterministic policy gradients (DPG) for continuous actions, A3C, Ape-X, R2D2, PPO, rainbow, universal value function approximation (UVFA), hindsight experience replay.

Deep reinforcement learning makes the headlines again when AlphaGo, a program combining RL and DL, manages to defeat the world champion in Go. Several extensions followed, namely, AlphaGo Zero and muZero.

Another important area for deep reinforcement learning is robotics, where researchers have applied it to robotic arms, mobile robots, drones, and autonomous cars. Similarly, recent advances have been obtained in natural language processing. The area requires the development of algorithms that use less resources (data and machines) and more general results that could be applied to different, although similar, tasks.

6.2.5 Other approaches

There is a long list of other machine learning techniques not mentioned in the previous sections. Here we very briefly mention some of them.

Most classifiers use a representation based on attribute-value pairs equivalent to propositional logic. There are other algorithms that use more expressive representations based on first-order logic. One of them is inductive logic programming (ILP) [24]. The benefits of a more expressive representation are that it allows including functions, variables, relations, and recursive definitions, which can produce more expressive and concise models.

A very popular technique that has shown clear improvements over traditional techniques is using ensembles [25]. The idea is to induce several models from the data and combine their results, which in general can produce better performance. In classifiers, two particular techniques that have shown very good results are random forest [5] and XGBoost [7].

Another technique that has become particularly relevant in recent years is transfer learning [26]. The idea of transfer learning is to use information from a previously solved task for solving a new task. Different elements can be transferred, such as data, models, or parameters. With transfer learning, a model for the new task can be induced faster, with less data, and possibly with higher performance than the model induced with only the original data. With the high data and computational demands required for deep neural networks, transfer learning has become a recurrent theme in this area.

In many cases, there is a large amount of data, but only a small proportion is labeled. This is common in text and images. An interesting research area is semi-supervised learning, where the idea is to use information from the unlabeled data to improve the induced model from the labeled data [41].

It is common to have data which changes its distribution over time and the induced models are no longer valid after a while. Concept drift tries to identify when an induced model is no longer valid and how to modify it, or when to induce a new model for the current data distribution [16].

There are many more algorithms and methods that have been developed in machine learning. Some of them have lost popularity in recent years, such as explanation-based learning, case-based reasoning, learning by analogy, learning by discovery, among many others. Some others are expected to become more relevant in the near future, such as life-long learning, where the idea is to have systems that keep their learning process continuous, that is, without interruption.

6.3 Conclusions

Machine learning, is at the core of the current hype on artificial intelligence. Vast amounts of money are being currently expended by nations and companies seeking economic, political, and social advantages over their peers. Seeking more intelligent systems currently resembles the arm race featuring the Soviet Union and the United States in the cold war period, but now between China and the United States to generate more intelligent systems.

The increasing automatization of processes in general, has been displacing jobs continuously, but never at the pace that it is expected to happen. There is also an increasing fear for the possible loss of jobs with more intelligent machines.

There are also concerns for an increasing delegation of decisions to machine (which are not very intelligent) that are trained with biases in the data and that are showing them in their decisions, threat-

ening, in many cases, human rights. Although several legislations are now appearing to mitigate this new phenomenon, the technology is growing much faster than the legislation.

Some researchers believe that given the current pace of development in machine learning, soon we will be able to create intelligent being with an intelligence superior to humans. Although it is very unlikely that this will happen in the near future, it is also irresponsible for us not to think about their possible consequences, and how to deal with it in a more, as S. Russell puts it [31], *human compatible* way.

References

- [1] R. Agrawal, R. Srikant, Fast algorithms for mining association rules, in: Proceedings of the 20th International Conference on Very Large Data Bases, Morgan Kaufmann, 1994, pp. 487–499.
- [2] E. Alpaydin, Introduction to Machine Learning, fourth edition, MIT Press, 2020.
- [3] C. Bishop, Pattern Recognition and Machine Learning, Springer, 2016.
- [4] B. Boser, I. Guyon, V. Vapnik, A training algorithm for optimal margin classification, in: Proceedings of the Fifth Annual Workshop of Computational Learning Theory (COLT), ACM, 1992, pp. 144–152.
- [5] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32.
- [6] L. Breiman, J. Friedman, C. Stone, R. Olshen, Classification and Regression Trees, Chapman & Hall/CRC, 1984.
- [7] T. Chen, C. Guestrin, Xgboost: a scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2016, pp. 785–794.
- [8] A. Feigenbaum, The simulation of verbal behavior, in: E.A. Feigenbaum, J. Feldman (Eds.), Computers and Thought, McGraw–Hill, 1963, pp. 297–309.
- [9] P. Flach, Machine Learning: The Art and Science of Algorithms That Make Sense of Data, Cambridge University Press, 2012.
- [10] M. García-Limón, H.J. Escalante, E.F. Morales, A. Morales-Reyes, Simultaneous generation of prototypes and features through genetic programming, in: D.V. Arnold (Ed.), Genetic and Evolutionary Computation Conference, GECCO’14, Vancouver, BC, Canada, July 12–16, 2014, ACM, 2014, pp. 517–524.
- [11] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.
- [12] I.J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, in: Advances in Neural Information Processing Systems (NIPS 27), Curran Associates, Inc., 2014, pp. 2672–2680.
- [13] G. Hinton, S. Osindero, Y. WhyeTeh, A fast learning algorithm for deep belief nets, Neural Computation 18 (7) (2006) 1527–1554.
- [14] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.
- [15] E.B. Hunt, C.I. Hoyland, Programming a model of human concept formation, in: E.A. Feigenbaum, J. Feldman (Eds.), Computers and Thought, McGraw–Hill, 1963, pp. 310–325.
- [16] A. Iwashita, J. Papa, An overview on concept drift learning, IEEE Access 7 (2019) 1532–1547.
- [17] A. Krizhevski, I. Sutskever, G. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems (NIPS 25), Curran Associates, Inc., 2012, pp. 1097–1105.
- [18] W.S. McCulloch, W. Pitts, A logical calculus of the ideas immanent in nervous activity, Bulletin of Mathematical Biophysics 5 (1943) 115–133.
- [19] D. Michie, R.A. Chambers, Boxes: an experiment in adaptive control, Machine Intelligence 2 (2) (1968) 137–152.

- [20] M. Minsky, S. Papert, *Perceptrons: An Introduction to Computational Geometry*, Cambridge University Press, 1969.
- [21] T.M. Mitchell, *Machine Learning*, McGraw-Hill, 1997.
- [22] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing Atari with deep reinforcement learning, arXiv preprint, arXiv:1312.5602, 2013.
- [23] E. Morales, Educando al 'niño computacional' de Turing, in: *Ciencias*, 2013, pp. 32–39.
- [24] S. Muggleton, Inductive logic programming, *New Generation Computing* 8 (1991) 295–318.
- [25] D. Opitz, R. Maclin, Popular ensemble methods: an empirical study, *Journal of Artificial Intelligence Research* 11 (1999) 169–198.
- [26] S.J. Pan, Q. Yang, et al., A survey on transfer learning, *IEEE Transactions on Knowledge and Data Engineering* 22 (10) (2010) 1345–1359.
- [27] J. Pearl, *Causality: Models, Reasoning and Inference*, 2nd edition, Cambridge University Press, 2009.
- [28] J.R. Quinlan, *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993.
- [29] F. Rosenblatt, The perceptron: a probabilistic model for the information storage and organization in the brain, *Psychological Review* 565 (6) (1958) 433–460.
- [30] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning internal representations by error-propagation, in: D.E. Rumelhart, J.L. McClelland (Eds.), *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, Vol. 1, MIT Press, 1986, pp. 318–362.
- [31] S. Russell, *Human Compatible: Artificial Intelligence and the Problem of Control*, Viking, 2019.
- [32] A.L. Samuel, Some studies in machine learning using the game of checkers, *IBM Journal of Research and Development* 3 (3) (1959) 210–229.
- [33] C. Shannon, W. Weaver, *The Mathematical Theory of Communication*, University of Illinois Press, 1949.
- [34] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, *Nature* 529 (7587) (2016) 484.
- [35] P. Spirtes, C. Glymour, An algorithm for fast recovery of sparse causal graphs, *Social Science Computer Review* 9 (1) (1991) 62–72.
- [36] R.S. Sutton, A.G. Barto, F. Bach, et al., *Reinforcement Learning: An Introduction*, second edition, MIT Press, 2018.
- [37] G. Tesauro, Temporal difference learning and td-gammon, *Communications of the ACM* 38 (3) (1995) 58–68.
- [38] A. Turing, Computing machinery and intelligence, *Mind* 59 (1950) 433–460.
- [39] C. Watkins, *Learning from Delayed Rewards*, PhD thesis, Cambridge University, Cambridge, England, 1989.
- [40] I. Witten, E. Frank, M. Hall, C. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, fourth edition, Morgan Kaufmann, 2017.
- [41] X. Zhu, Semi-supervised learning literature survey, *Computer Science*, University of Wisconsin-Madison 2 (3) (2006) 4.